UNIVERSIDAD TECNOLOGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Trabajo Final Integrador

Dog Walk App

Plataforma de paseos grupales y reservas por mascota en CABA

Integrantes

Gastón Lell

Gabriel Lovera

Juan Cruz Leal

Noviembre de 2026

Índice

Sección	Página
Resumen ejecutivo y planteamiento	3
1. Alcance y requisitos	4
2. Arquitectura y módulos	5
3. Modelo de datos	6
Relaciones entre las tablas	7
4. Reglas de negocio y seguridad	8
5. Plan de validación y resultados	9
6. Evidencias: cobertura y publicación	10
7. Evidencias: mascotas y búsqueda	11
8. Evidencias: reserva y cancelación	12
9. Evidencias: finalización y reseña	13
10. Instalación y recorrido de demostración	14
11. Organización, límites y conclusiones	15
Anexo: contrato de API y fuentes	16
Anexo: capturas adicionales	17

Resumen ejecutivo y planteamiento

Dog Walk App es una aplicación web para organizar paseos grupales de perros en la Ciudad Autónoma de Buenos Aires. El paseador publica el barrio, el horario, el punto de encuentro, el precio y el cupo. El dueño registra sus mascotas y puede inscribirlas de a una. Cada mascota ocupa un lugar. En el proyecto también usamos el nombre Paseadores.

El paseador configura su cobertura, publica salidas y consulta su agenda. El dueño puede agregar una mascota al registrarse, buscar por barrio, día u horario y reservar para una de sus mascotas. Mis reservas separa las salidas en Próximas, En curso e Historial. Desde el historial, si participó de un paseo finalizado, puede calificarlo.

Problema que nos propusimos resolver

Cuando una salida se organiza por mensajes, el horario, el punto de encuentro y los cambios de último momento pueden quedar repartidos en varias conversaciones. También es difícil saber cuántas mascotas ya se anotaron. Decidimos reunir esos datos en la ficha del paseo y darles a ambos roles un lugar para consultar lo que tienen programado.

Objetivo general

Desarrollar una aplicación web con persistencia relacional que permita organizar paseos por barrio y confirmar reservas de acuerdo con el cupo real de cada salida.

Objetivos específicos

- Permitir que un paseador publique en los barrios que indicó como cobertura.
- Dar al dueño una búsqueda por barrio y una forma de reservar para sus propias mascotas.
- Mantener el historial y respetar los permisos al cancelar, completar o reseñar una salida.
- Verificar con dos solicitudes simultáneas que el último lugar solo se confirme una vez.

Probamos el recorrido de ambos roles con cuentas ficticias y una base local. Mostramos las pantallas y los resultados obtenidos. Los enlaces de la aplicación publicada y del video se incorporarán cuando estén disponibles.

1. Alcance y requisitos

Definimos dos roles, dueño de mascota y paseador. Cada uno tiene opciones de navegación propias. El catálogo de barrios es abierto; las pantallas que usan datos de cuenta, paseos o reservas requieren iniciar sesión.

Requisito	Comportamiento esperado
RF01. Cuenta y perfil	Crear una cuenta con uno de los dos roles, iniciar sesión y editar los datos propios. El perfil que otros usuarios pueden ver no expone el teléfono ni el correo del paseador.
RF02. Cobertura	Guardar barrios y valores de referencia para el precio y el cupo. El formulario de publicación usa esa cobertura.
RF03. Publicación	Crear una salida futura con duración, precio, cupo y encuentro. El paseador recibe una confirmación y puede ver la salida en su agenda. Un horario superpuesto se rechaza.
RF04. Mascotas y búsqueda	Agregar y editar mascotas propias. Buscar paseos futuros por barrio, día u horario y consultar el detalle.
RF05. Reserva	Inscribir una mascota propia por solicitud si queda lugar y no llegó la hora de inicio. Consultar Próximas, En curso e Historial.
RF06. Cancelación y cierre	Liberar el cupo al cancelar una reserva a tiempo. El paseador puede iniciar con mascotas inscriptas y finalizar un paseo que está en curso.
RF07. Reseñas	Aceptar una calificación por dueño participante y paseo finalizado. Mostrar la reseña y actualizar la valoración del paseador.

Criterios de calidad

Validamos permisos y disponibilidad en el servidor, usamos restricciones en la base para evitar duplicados y cuidamos que la interfaz pueda usarse en pantallas pequeñas. La guía de instalación y la prueba con los dos roles permiten repetir el recorrido con una base nueva.

Límites del alcance

Los precios son informativos y están en pesos argentinos. El mapa ubica barrios de forma orientativa y el encuentro se describe con texto. Esta versión no incluye cobros en línea, ubicación en tiempo real, chat ni aplicación móvil nativa.

2. Arquitectura y módulos

La interfaz usa React, TypeScript y Vite. Spring Boot aplica las reglas de la API y PostgreSQL almacena los datos. Flyway ejecuta las migraciones. En desarrollo, Vite reenvía /api al backend. Para producción, el Dockerfile compila la interfaz y la empaqueta junto con Spring Boot en un solo servicio.

Capa	Responsabilidad
Interfaz	Pantallas por función: ingreso, perfil, mascotas, búsqueda, detalle, agenda, reservas y reseñas. Muestra el resultado y los mensajes de validación.
Controladores	Reciben las solicitudes HTTP, identifican al usuario de la sesión y convierten los resultados o errores del servicio en respuestas de la API.
Servicios	Deciden si una operación está permitida. Por ejemplo, controlan cobertura, solapamientos, propiedad de una mascota y cupo antes de reservar.
Persistencia	Los repositorios consultan PostgreSQL y guardan las entidades. Las restricciones y transacciones resuelven condiciones que la interfaz no puede garantizar.

Organización del repositorio

Directorio	Contenido
frontend/	Pantallas, rutas, formularios y cliente HTTP.
backend/	Controladores, servicios, entidades, repositorios, migraciones y pruebas de Java.
database/	DDL, datos iniciales, consultas de comprobación y esquema editable DBML.
docs/	Propuesta, decisiones, instrucciones, contrato de la API y evidencias.

Decisión de implementación

El módulo de paseos agrupa publicación, búsqueda y agenda. BookingService coordina la inscripción y la cancelación, y WalkPetService comprueba las reglas de cada reserva. Al publicar una salida, el paseador recibe una confirmación con un enlace para compartir. La agenda separa las próximas salidas del historial.

3. Modelo de datos

La base es relacional. El siguiente esquema resume las relaciones que usa la aplicación. Las claves y los tipos exactos están en las migraciones de Flyway; el DBML del repositorio permite editar el diseño.

Tabla	Relación y datos principales
zones	Catálogo de barrios. Cada barrio puede estar asociado a usuarios, paseos y coberturas.
users	Cuenta de dueño (**CLIENT**) o paseador (**WALKER**), barrio de referencia y datos del perfil. El paseador también puede tener precio, cupo y valoración de referencia.
walker_zones	Vincula un paseador con cada barrio en el que ofrece sus servicios.
pets	Cada mascota pertenece a un dueño. Guarda nombre, tamaño, raza y notas según corresponda.
walks	Cada salida pertenece a un paseador y un barrio. Guarda horario, duración, punto de encuentro, precio, cupo y estado.
walk_pets	Vincula una mascota con un paseo. Tiene un estado propio para conservar cancelaciones y permitir reactivar la misma reserva.
reviews	Cada reseña corresponde a un paseo y a un dueño. Guarda puntuación, comentario y fecha.

Restricciones de integridad

El correo es único sin distinguir mayúsculas de minúsculas. También son únicos los pares paseador/barrio, paseo/mascota y paseo/autor. Las claves foráneas evitan referencias a registros inexistentes; las restricciones de la base exigen cupos y duraciones positivos, precios no negativos y calificaciones de uno a cinco.

Datos iniciales

La primera migración crea el esquema y la segunda carga los barrios. Para la demostración usamos una base vacía y generamos desde la aplicación las cuentas, las mascotas y los paseos. El precio se almacena como decimal y los horarios se interpretan en la zona horaria de Buenos Aires.

Material del diseño: modelo de datos y diagrama del repositorio.

Relaciones entre las tablas

Cada flecha parte de la clave primaria de una tabla y termina en el campo que la referencia. Incluimos solo las claves y algunos atributos de uso frecuente; la definición completa está en la primera migración.

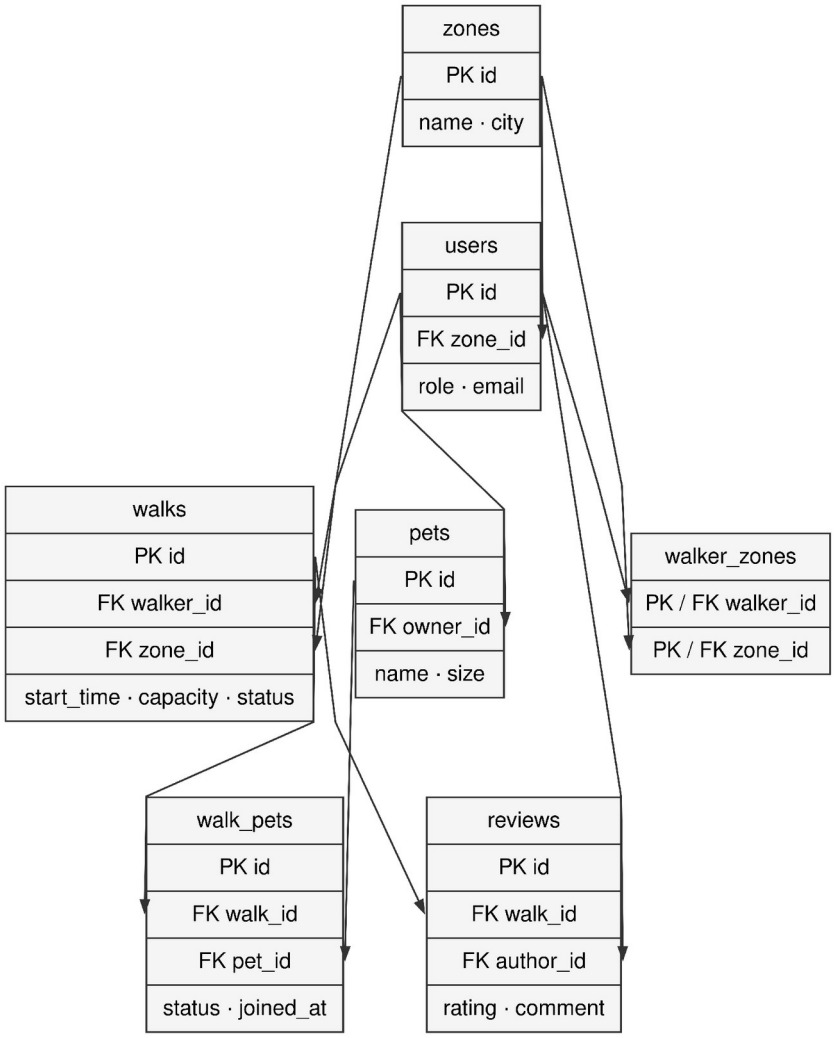


Diagrama relacional simplificado a partir del esquema SQL. PK: clave primaria. FK: clave foránea.

Cómo se usan estas relaciones

Un usuario tiene un barrio de referencia. El paseador puede, además, registrar varios barrios de cobertura. Cada paseo se vincula con el paseador que lo publica y con el barrio donde se realiza.

Una mascota puede anotarse en distintos paseos, y cada paseo puede tener varias mascotas. La tabla walk_pets representa esa relación y guarda el estado de cada reserva. El dueño califica el paseo en reviews, una sola vez aunque haya anotado más de una mascota.

4. Reglas de negocio y seguridad

Publicación y agenda

Para publicar, el paseador necesita un barrio de su cobertura, un inicio con al menos dos horas de anticipación y una duración de 30 minutos o más, en múltiplos de 15. El servicio toma un bloqueo por paseador y rechaza los horarios que se superponen.

Reserva y cancelación

Cada reserva inscribe a una mascota propia. El backend bloquea la fila del paseo y, dentro de la misma transacción, comprueba que siga publicado, que no haya llegado la hora de inicio y que quede lugar. Si dos personas solicitan el último cupo, la segunda espera y comprueba nuevamente la disponibilidad.

El dueño puede cancelar si el paseo sigue programado y faltan más de dos horas. La reserva queda en el historial y libera un lugar; si vuelve a inscribir a la misma mascota, se reactiva el registro. Cuando el paseador cancela una salida, también se cancelan las reservas activas.

Estados del paseo y reseñas

Estado	Cuándo se usa
Publicado o completo	Salida programada. El estado cambia automáticamente al llenarse o liberarse un lugar.
En curso	El paseador puede iniciar una salida publicada o completa si tiene al menos una mascota inscripta.
Finalizado	El paseador puede finalizar una salida en curso. Un cliente que participó ya puede reseñarla.
Cancelado	El paseador puede cancelar una salida publicada, completa o en curso. Se cancelan las reservas activas y se conserva el historial.

Solo un dueño que participó puede reseñar un paseo finalizado, una vez. Al guardar la reseña, el servicio actualiza el promedio y la cantidad de valoraciones del paseador.

Control de acceso

La API identifica al usuario con JWT y guarda las contraseñas con hash BCrypt. La búsqueda, el detalle y el perfil de otros paseadores requieren sesión. El perfil visible para otros usuarios no expone su correo ni su teléfono. Las operaciones de escritura comprueban rol y propiedad en el servidor. Las credenciales de la base de datos y la clave JWT se leen de la configuración del entorno.

5. Plan de validación y resultados

Probamos con Java 21, PostgreSQL 17 y Node.js 24. Usamos una cuenta de paseador y otra de dueño. Recorrimos las pantallas del cliente a 390 px de ancho.

Comprobación	Resultado observado
Backend	211 pruebas aprobadas con Java 21 y PostgreSQL 17, incluidas las de integración.
Frontend	51 pruebas aprobadas. También pasaron la compilación y el lint.
Solapamiento	Dos publicaciones simultáneas con horarios superpuestos no se guardaron juntas.
Último cupo	En dos solicitudes simultáneas para el último lugar, solo se confirmó una.
Permisos y estados	Verificamos cancelación, reinscripción, transiciones y rechazo al llegar la hora de inicio.
Navegador y móvil	Recorrimos ambos roles y validamos búsqueda, reservas y reseñas a 390 px de ancho.

Sobre las capturas y las pruebas

Con cuentas ficticias comprobamos la publicación, la reserva del último lugar, la cancelación y la reinscripción. También vimos cómo pasa la reserva de Próximas a En curso y después a Historial, desde donde enviamos la reseña.

Comportamiento de las reservas

La búsqueda y la inscripción dejan de ofrecer el paseo al llegar su hora de inicio. En Mis reservas, la pestaña En curso sigue el estado que indica el paseador. Los paseos finalizados y las reservas canceladas aparecen en el historial.

6. Evidencias: cobertura y publicación

Configuramos Palermo como barrio de cobertura del paseador, con un lugar y un precio de referencia. Publicamos una salida y comprobamos que apareciera en su agenda. Al intentar crear otra en el mismo horario, la API rechazó la publicación.

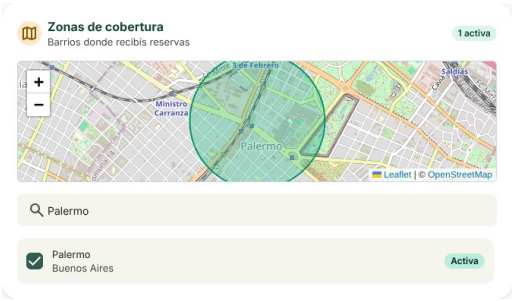


Figura 1. Palermo en la cobertura del paseador.



Figura 2. Confirmación al publicar la salida.

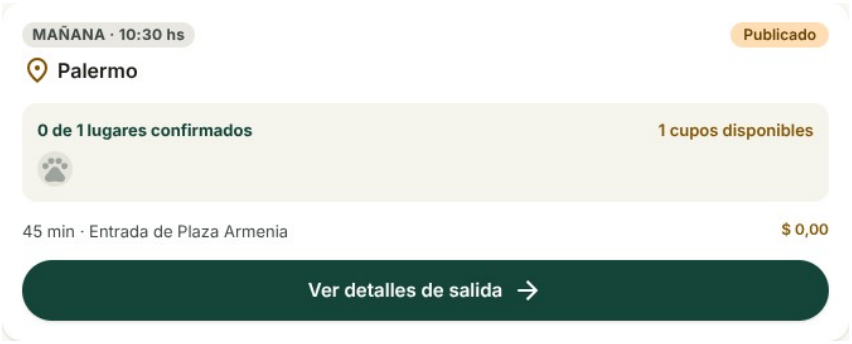


Figura 3. Salida publicada en la agenda del paseador, con horario, cupo y punto de encuentro.

7. Evidencias: mascotas y búsqueda

Registramos a Sofía y agregamos a Luna en el segundo paso. Buscamos por barrio y día; la tarjeta del paseo mostraba el horario, el encuentro y el lugar disponible.



Figura 4. Luna aparece en Mis mascotas.



Figura 5. Resultado en Palermo con el horario, el precio y un lugar disponible.

8. Evidencias: reserva y cancelación

Reservamos el único lugar para Luna. Cancelamos desde Próximas, vimos la reserva en Historial y comprobamos que el lugar se liberara. Después volvimos a reservar para la misma mascota.



Figura 6. El detalle muestra el único lugar ocupado.



Figura 7. Reserva confirmada en Próximas, en la cuenta de la dueña.



Figura 8. La reserva cancelada permanece en el historial.

9. Evidencias: finalización y reseña

Después de volver a reservar, el paseador inició y finalizó la salida. En el historial de la dueña elegimos Calificar el paseo, enviamos cinco estrellas y comprobamos la valoración en el perfil del paseador.

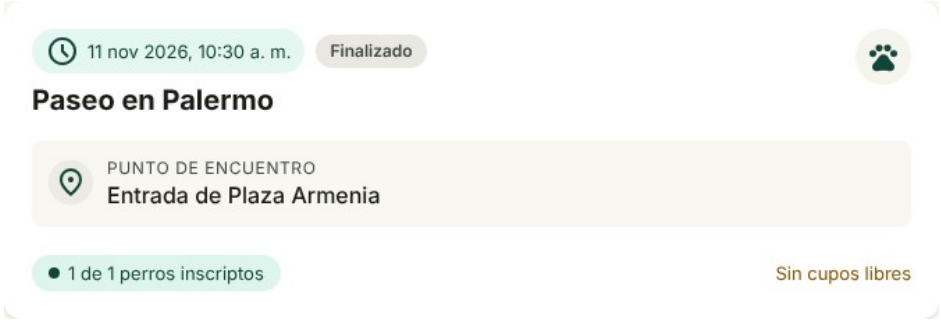


Figura 9. El paseo figura como finalizado en la vista del paseador.



Figura 10. El perfil muestra el promedio de 5 sobre 5 y una reseña.



Figura 11. La reseña publicada se puede leer desde una pantalla de 390 px.

Los nombres y las mascotas son ficticios. El punto de encuentro se usó como ejemplo.

10. Instalación y recorrido de demostración

Base de datos y configuración

Crear una base en PostgreSQL 14 o posterior. Para ejecutar el backend necesitamos Java 21 y una configuración local de conexión. Flyway crea el esquema y carga los barrios al arrancar.

```
DB_URL=jdbc:postgresql://localhost:5432/paseadores
DB_USER, DB_PASSWORD y JWT_SECRET: valores locales.
```

Las variables se pueden exportar o guardar en backend/.env, fuera del repositorio. El Dockerfile de la raíz empaqueta frontend y backend en una imagen; PostgreSQL se configura como servicio separado. La guía del repositorio incluye ambos modos de ejecución.

Backend y frontend en terminales separadas

Desde la raíz, abrir dos terminales. En la primera, ejecutar el backend con el wrapper de Maven. En la segunda, instalar las dependencias con Node.js 24 y pnpm 11.25.0, y arrancar Vite.

```
cd backend && bash mvnw spring-boot:run
```

```
cd frontend && pnpm install --frozen-lockfile && pnpm dev
```

En Windows se puede usar mvnw.cmd. Abrir http://localhost:5173. Vite reenvía /api a Spring Boot en http://localhost:8080. Swagger está disponible en /swagger-ui/index.html en el backend.

Pasos para repetir la demostración

1. Crear un paseador, agregar Palermo a su cobertura y publicar una salida.
2. Registrar un dueño, agregar una mascota, buscar por barrio y día y reservar desde el detalle.
3. Cancelar en Próximas con más de dos horas de anticipación y volver a reservar.
4. Iniciar el paseo y comprobar En curso. Finalizarlo y, desde Historial, elegir Calificar el paseo.

11. Organización, límites y conclusiones

Organización del trabajo

Organizamos las tareas en ClickUp y trabajamos en conjunto en GitHub. Juan Cruz se encargó del análisis y la integración de los módulos, Gabriel trabajó en el esquema y los scripts de base de datos, y Gastón organizó la estructura del proyecto. Luego seguimos entre los tres con las funcionalidades, las pruebas y los ajustes de la aplicación.

Trabajamos sobre el recorrido del cliente, las reservas y la agenda del paseador. Usamos la propuesta de Stitch para la búsqueda, el detalle y las reservas del cliente. El paseador conserva su perfil y sus pantallas de agenda.

Antes de la entrega

Pendiente	Comprobación para cerrarlo
Revisión del equipo	Revisar el informe entre todos y confirmar el recorrido que se va a mostrar en la entrega.
Despliegue público	Publicar la aplicación con una base PostgreSQL persistente y comprobar que se pueda ingresar y reservar desde el enlace público.
Video de demostración	Grabar el recorrido de ambos roles con datos ficticios, comprobar que el enlace funcione e incorporarlo al README y al informe.

Conclusiones

Durante el desarrollo vimos que mostrar los lugares libres en pantalla era solo una parte del problema. Lo más importante ocurría al confirmar: si dos personas pedían el mismo cupo, la base tenía que aceptar una sola. La prueba concurrente nos sirvió para comprobar esa regla con dos solicitudes separadas.

Probamos que el cupo y los estados se reflejen en ambas cuentas, y que el cliente pueda calificar desde su historial. Para cerrar la entrega todavía debemos verificar el funcionamiento en el despliegue público y agregar el enlace al video.

Anexo: contrato de API y fuentes

La API usa JSON y un token Bearer en las operaciones protegidas. Swagger está en /swagger-ui/index.html. Estas son las rutas principales del recorrido.

Método y ruta	Operación
POST /api/auth/register POST /api/auth/login	Crear cuenta e iniciar sesión.
GET /api/zones GET / PUT /api/walkers/me	Consultar catálogo y administrar la cobertura propia.
GET / POST /api/walks GET /api/walks/mine	Buscar o publicar paseos y consultar la agenda del paseador.
POST /api/walks/{id}/join/{petId} GET /api/walks/bookings/mine	Inscribir una mascota propia. Consultar Próximas (UPCOMING), En curso (IN_PROGRESS) o Historial (HISTORY).
DELETE /api/walks/{id}/join/{petId}	Cancelar una reserva propia.
PATCH /api/walks/{id}/status POST /api/walks/{id}/reviews	Cambiar el estado de una salida o calificar un paseo finalizado como dueño participante.

Fuentes y materiales

Repositorio del equipo: https://github.com/loveragabriel/trabajo-practico-integrador

Propuesta, arquitectura y alcance

Controlador de paseos y reservas

Guía de ejecución local

Universidad Tecnológica Nacional, Manual de identidad visual

OpenStreetMap, atribución de datos cartográficos

Anexo: capturas adicionales

El paseador puede ver el estado y las mascotas inscriptas. Cuando inicia la salida, la dueña la encuentra en En curso.



Figura 12. Estado En curso en el detalle del paseador.

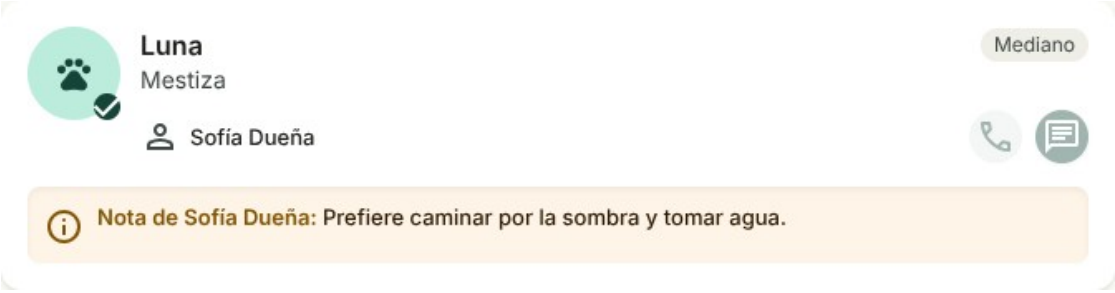


Figura 13. Mascota inscripta y observaciones en el detalle del paseador.



Figura 14. La dueña encuentra su reserva en En curso.